

Enterprise Event-Driven Architecture: Real-World Apache Kafka Applications, Spring Kafka Patterns, and Practical Scenarios

Real-World Enterprise Implementations of Apache Kafka

Modern distributed architectures are shifting away from synchronous, blocking HTTP communication because direct inter-service REST calls bind systems together, creating brittle distributed monoliths¹. Enterprise platforms resolve this by using Apache Kafka as a high-throughput, fault-tolerant message bus². This transition from synchronous communication to asynchronous, event-driven design allows systems to decouple transactional paths, scale components independently, and ensure high availability².

Uber: High-Throughput Processing and Asynchronous Consumer Proxies

At Uber, Apache Kafka is a core component of the real-time processing stack, handling trillions of messages and petabytes of data daily². Kafka supports many business operations, including rider-driver matching, dynamic surge pricing, ad event tracking, and fraud detection².

Because Uber has scaled its pub-sub messaging system to process over 12 million messages per second, standard client polling patterns presented partition scalability issues and head-of-line blocking³. In standard Kafka consumer design, partition message ordering requires that a consumer instance sequentially process and acknowledge a message before fetching the next record from that partition³.

For asynchronous processing tasks, such as payment gateway integration, this design can lead to system-wide bottlenecks³. If the billing service experiences a latency spike when communicating with a payment provider (e.g., Visa), the consumer thread assigned to that partition blocks³. This delay halts the processing of all other events in the same partition, including those for other payment providers (e.g., Mastercard), even if those downstream services are healthy³.

To solve this problem, Uber developed uForwarder, a specialized consumer proxy layer². Rather than running native Kafka consumer code inside each of its hundreds of microservices, Uber routes consumer traffic through this proxy cluster³. The uForwarder system fetches messages from the Kafka brokers using the standard binary protocol and pushes individual events to consumer microservices via gRPC endpoints³.

This design separates partition offset management from the business execution thread pool³.

Downstream instances process messages concurrently and send execution acknowledgments back to the proxy, which handles committing offsets². This mechanism prevents slow, high-latency tasks from blocking healthy event streams³.

For low-latency transactions, such as financial updates to user balances, Uber's User Account Batch Processing engine bypassed client-side Kafka batching⁷. Traditional Kafka processing was rejected for this specific transactional path because normal consumer-side poll delays of 1 to 2 seconds were too high for real-time account authorization⁷.

Instead, Uber built a sub-second batching engine that groups operations into 250-millisecond windows using Redis-based coordination⁷. A batch processor reads the current user account state from database storage, applies the grouped batch in-memory, writes tentative audit logs to an asynchronous post-processing service, and performs an atomic write back to the database using optimistic concurrency control⁷.

The decoupled post-processing service then reads these tentative logs from database storage and publishes the finalized transaction records to Kafka⁷. This pattern illustrates how highly transactional, sub-second systems can utilize database-backed batching for state updates while relying on Kafka to broadcast downstream audit trails asynchronously⁷.

Airbnb: Low-Latency Personalization and SpinalTap Change Data Capture

Airbnb operates Apache Kafka across massive streaming pipelines, ingesting over 35 billion events per day⁸. The data pipeline is divided into two primary execution models: analytical data ingestion and real-time operational flows⁸.

Analytical event logging leverages Airstream, an internal Spark Streaming framework, to read from Kafka, deduplicate events against HBase, and write them to hourly Hive partitions⁸.

However, Spark's micro-batch model is too slow for real-time personalization, which relies on sub-second latency⁸.

To support real-time personalization, Airbnb built the User Signals Platform (USP), which handles over 70,000 queries per second⁸. When users interact with the Airbnb website or mobile app (viewing listings, searching, or booking), events flow into Kafka⁸.

The USP deploys over 100 Apache Flink jobs that consume these events event-by-event, bypassing Spark's micro-batch latency⁸. These Flink jobs aggregate event metrics, compute structured user signals, write the outputs to a low-latency key-value store, and publish the enriched signals back to Kafka for consumption by downstream targeting engines⁸.

To capture database mutations and propagate them across search engines and caches without blocking the primary request write path, Airbnb developed and open-sourced SpinalTap, a Change Data Capture (CDC) engine⁸. SpinalTap captures mutations from MySQL, DynamoDB, and internal storage platforms and publishes them as standardized Kafka events⁸.

These CDC event streams are serialized using Apache Thrift to support both Java and Ruby consumer microservices⁸. SpinalTap feeds these database change logs into multiple downstream systems simultaneously:

Downstream Target	Primary Infrastructure	Business Impact
Search Indexing	Elasticsearch	Updates listing and inbox search indexes based on database changes ⁸ .
Cache Invalidation	Memcached, Redis	Evicts stale records directly from cache layers without impacting write queries ⁸ .
Service Signaling	Microservices	Triggers downstream workflows (e.g., notifying the Availability service of booking events) ⁸ .

By implementing logical epoch-based split-brain mitigation and preserving commit-order sequencing, SpinalTap ensures a durable, zero-data-loss replication loop that maintains eventual consistency across Airbnb's databases, caches, and indexes⁸.

DoorDash: Schema-Validated Ingest and Asynchronous Inventory Workflows

DoorDash utilizes Apache Kafka as its main streaming fabric, leveraging several architectural optimizations to handle high-volume write workloads¹⁰. DoorDash designed an open-source-based Kafka REST Proxy as a centralized gateway to handle event production¹⁰. This REST Proxy provides an HTTP interface for client applications, abstracting away broker connection pools¹⁰. The proxy performs schema validation on incoming JSON payloads against the Confluent Schema Registry and packages events into highly compressed binary formats¹⁰. DoorDash customized this REST proxy to support asynchronous producer requests, returning immediate HTTP success responses to client systems as soon as the payload is validated and written to the proxy's local memory buffer, rather than waiting for broker acknowledgment¹⁰. While this asynchronous approach introduces minor risks of data loss, DoorDash mitigates these through client-side retries and broker metric tracking¹⁰. To optimize broker performance, DoorDash uses a replication factor of two with one minimum in-sync replica¹⁰. The producers are configured to require acknowledgment from the partition leader only, rather than all replicas, and utilize sticky partitioning with a linger window of 50ms to 100ms¹⁰. This batch optimization reduced average broker CPU utilization by 30% to 40%¹⁰. On the operational application layer, DoorDash relies on this streaming pipeline to run its write-heavy Inventory Platform¹¹. When merchants upload flat inventory files, or when delivery drivers submit live stock signals through their mobile applications while shopping, the data is processed asynchronously¹¹.

The ingestion system saves raw files directly to AWS S3, while inventory workflows are executed asynchronously using Cadence, a microservices orchestration engine¹¹. Raw item mutations are pushed to a Kafka layer, consumed by a Apache Flink job, and loaded into Snowflake for analytical queries¹¹.

Simultaneously, a predictive model monitors historical item-not-found (INF) logs to calculate real-time item availability and update customer-facing store pages, avoiding expensive relational database writes during peak ordering hours¹¹.

For financial audit and ledger processing, DoorDash utilizes its Archiver microservice, a Flink application that consumes multiple financial event streams from Kafka¹².

The Archiver groups, encodes, and flushes transactional logs to AWS S3 in Apache Parquet format, enabling auditing and correction engines to query historical events using Snowflake without impacting live production database performance¹².

Expedia: Scale WebSocket Processing and SLO Metric Consolidation

Expedia Group uses event-driven architectures to power live user experiences and track system reliability¹³. Expedia engineered a real-time event exploration platform capable of streaming approximately 3 million records per minute to customer web browsers¹³.

The primary challenge of streaming events directly from Kafka to browsers via WebSockets (using SockJS and STOMP) is the resource overhead of maintaining millions of concurrent, open socket connections and filtering high-velocity topics on the fly¹³.

Expedia resolved this by decoupling event filtering from WebSocket session management¹³. They deployed a Filter Worker microservice in Kubernetes that scales via standard consumer groups to process high-volume Kafka topics¹³.

When a user registers an event query on the UI, the browser registers the filter criteria in a PostgreSQL database¹³. This database uses triggers and standard notifications (NOTIFY notify_filter_changes()) to alert the Filter Workers to dynamically load the new filters¹³.

The Filter Workers consume the high-volume source streams, apply the active user filters, and write the matching records to a much smaller, partitioned "filtered" Kafka topic¹³. The partition key for this filtered topic is the unique WebSocket session ID¹³.

The WebSocket Handler pods subscribe only to this filtered topic, consuming pre-sorted messages and pushing them to the appropriate browser connections¹³. This architecture separates concerns, enforces backpressure, and reduces running costs by 50% compared to direct-streaming designs¹³.

Expedia also uses Kafka to power its Service Level Objective (SLO) platform¹⁴. The data ingestion layer runs a Spring Boot microservice that periodically queries performance metrics from DataDog and writes them in raw format to a Kafka topic¹⁴.

A KStream streaming application consumes this raw stream, transforms the data to match internal governance schemas, enriches the records with application metadata, and writes the finalized events to a PostgreSQL database for historical trend analysis¹⁴.

This decoupled design extends metric retention from DataDog's 90-day limit to one year, bypasses DataDog API rate limits, and provides real-time SLO alert monitoring¹⁴.

Walmart: Optimizing Large Catalog Payloads and Event Scheduling

Walmart operates at an immense global scale, where even minor serialization and transmission overheads can accumulate into major system bottlenecks¹⁵. The Walmart Rollups platform aggregates catalog information from various sources to serve product pages¹⁵.

These catalog payloads frequently exceed several megabytes, creating network transmission bottlenecks and heavy garbage collection overhead in search JVMs during high-traffic periods¹⁵.

To optimize this process, Walmart decoupled catalog generation:

- The Rollups application operates as an offline Kafka consumer, aggregating catalog details and compressing the resulting JSON strings using the ZSTD compression algorithm¹⁵.
- These compressed ZSTD bytes are saved directly to a Cassandra database¹⁵.
- When a customer performs a search, the search service retrieves the compressed bytes from Cassandra and passes them to the client-side orchestrator¹⁵.
- The orchestrator uses pre-compiled Protocol Buffers (Protobuf) to decompress and deserialize the payload locally¹⁵.

Using Protobuf with varint encoding significantly reduces integer and field size on the wire compared to raw text-based JSON, minimizing network latency and memory overhead¹⁵.

For scheduling price updates and promotions, Walmart engineered a distributed, fault-tolerant event scheduler¹⁶. The scheduler accepts delayed requests via HTTP or Kafka¹⁶.

It calculates a time-bucket partition key (e.g., grouping events scheduled for 10:21:55 AM into a 10:21:00 AM bucket) and writes the scheduled tasks to a Cassandra cluster with tunable consistency (using EACH_QUORUM for critical promotions to prevent data loss)¹⁶.

A background Bucket Loader and Status Syncer rate-limits and loads active buckets into memory, executing the promotional updates across services at the scheduled second¹⁶.

To build these streaming microservices, Walmart leverages Spring Cloud Stream, using its binder abstraction layer to declaratively configure Kafka consumer groups, concurrency, and retry parameters within standard configuration files, removing boilerplate polling loops⁴.

Technical Mechanics of Spring Kafka Integration

Integrating Spring Boot with Apache Kafka allows development teams to build scalable streaming pipelines while utilizing declarative annotations and automatic configuration templates¹⁷.

Initialization Lifecycle and Configuration

Spring Boot simplifies integration with Kafka by auto-configuring core connection templates, serialization adapters, and error-handling interceptors¹⁷.

The primary configuration properties are managed within the application.yml file, specifying connection coordinates, group properties, and default serializer/deserializer classes¹⁷:

YAML

spring:

kafka:

bootstrap-servers: localhost:9092 # Connection coordinates for Kafka brokers

producer:

key-serializer: org.apache.kafka.common.serialization.StringSerializer cite: 17

value-serializer: org.springframework.kafka.support.serializer.JsonSerializer cite: 17

consumer:

group-id: inventory-service-group cite: 17 18

auto-offset-reset: earliest # Determines partition start behavior for new consumers [cite: 18]

key-deserializer: org.apache.kafka.common.serialization.StringDeserializer cite: 17

value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer

properties:

Prevents poison pills by separating deserialization logic [cite: 20]

spring.deserializer.value.delegate.class:

org.springframework.kafka.support.serializer.JsonDeserializer

spring.json.trusted.packages: "com.example.events"

To send and receive events, Spring Kafka provides the `KafkaTemplate` class and the `@KafkaListener` annotation¹. `@KafkaListener` abstracts away consumer polling mechanics, automatically managing consumer thread lifecycles and partition assignment¹:

Java

@Service

public class OrderEventService

private final KafkaTemplate<String, Object> kafkaTemplate

public OrderEventService(KafkaTemplate<String, Object> kafkaTemplate)

this.kafkaTemplate = kafkaTemplate

public void publishOrder(OrderCreatedEvent event)

```

// Asynchronously publish to topic using orderId as partition key
kafkaTemplate.send("orders.v1", event, orderId(), event)
    .whenComplete((result, exception) ->
        if (exception == null)
            System.out.println("Published event offset: " +
                result.getRecordMetadata().offset());
        else
            System.err.println("Publishing failed: " + exception.getMessage());
    );
}
}

@Slf4j
@KafkaListener(topics = "orders.v1", groupId = "fulfillment-group")
public void processOrder(OrderCreatedEvent event) {
    System.out.println("Processing order event: " + event.orderId());
}
}

```

Event Schemas: Deep Envelopes vs. Arbitrary Payloads

When designing event payloads for large microservice networks, organizations must choose between deep envelopes and arbitrary payloads to balance schema evolution and decoupling²¹:

```

- Deep Envelope Schema (Schema on Write)
+-----+
| Envelope Headers |
| (ID, Timestamp, Priority, Type) |
+-----+
| Explicit Payloads |
| (OrderCreated, OrderCancelled) |
+-----+

- Arbitrary Payload Schema (Schema on Read)
+-----+
| Envelope Headers |
| (ID, Timestamp, Priority, Type) |
+-----+

```



- **Deep Envelope Approach:** The envelope explicitly defines and packages all known payload models within a unified schema structure (schema on write)²¹.
 - *Advantages:* Common metadata fields (such as timestamps, IDs, and priorities) are standardized at the top level, allowing routers and decoders to handle events without deserializing the inner payload²¹. This ensures strong compile-time type safety and strict schema validation²¹.
 - *Disadvantages:* Tight coupling. Any payload schema update requires modifying the top-level envelope schema, which can slow down deployment timelines across teams and increase CPU overhead when parsing complex schemas²¹.
- **Arbitrary Payload Approach:** The envelope defines common metadata fields but packages the actual event payload inside an un-typed byte block or generic key-value map²¹.
 - *Advantages:* Maximum flexibility²¹. Individual services can publish new payload types without modifying the core envelope model, simplifying schema evolution and reducing deployment dependencies²¹.
 - *Disadvantages:* Weak validation²¹. Schema checks are deferred to runtime, increasing the risk of deserialization errors and requiring consumer applications to write custom parsing code to unpack arbitrary payloads²¹.

Resolving the Dual Write Problem with the Transactional Outbox Pattern

A common issue in event-driven systems is the "dual write" anti-pattern, where an application attempts to save state to a local database and publish a corresponding event to Kafka in the same execution path²².

If the database write succeeds but the network call to Kafka fails (due to broker timeouts, partitions rebalancing, or network issues), the downstream services will miss the state change, leading to data inconsistency²².

The **Transactional Outbox Pattern** solves this consistency challenge by utilizing local database transactions²².

Instead of publishing directly to Kafka, the application writes both the business entity updates and a generic outbox record to an outbox database table within the same ACID transaction²².


```

Java
@Service
public class OrderFulfillmentService

    private final OrderRepository orderRepository;
    private final OutboxRepository outboxRepository;
    private final ObjectMapper objectMapper;

    public OrderFulfillmentService(OrderRepository orderRepository,
                                    OutboxRepository outboxRepository,
                                    ObjectMapper objectMapper) {
        this.orderRepository = orderRepository;
        this.outboxRepository = outboxRepository;
        this.objectMapper = objectMapper;
    }

    @Transactional
    public void confirmOrder(UUID orderId) throws Exception {
        Order order = orderRepository.findById(orderId).orElseThrow();
        order.confirm(); // Perform state transition

        // Prepare the outbox record
        OutboxEvent event = new OutboxEvent(
            UUID.randomUUID(),
            "Order",
            order.getId().toString(),
            "OrderConfirmed",
            objectMapper.writeValueAsString(order)
        );

        orderRepository.save(order); // Save business state
        outboxRepository.save(event); // Save event in the same transaction block
    }

```

A separate Change Data Capture (CDC) system, such as Debezium, scans the database transaction logs (PostgreSQL WAL or MySQL Binlog)²².

When Debezium detects a commit to the outbox table, it extracts the payload and publishes it as a Kafka event²².

To optimize message routing, Debezium uses a Single Message Transform (SMT) to map fields dynamically²²:

- **Key Selection:** The `aggregate_id` is mapped to the Kafka record key, ensuring that all events for a given entity are routed to the same partition to maintain strict ordering²².

- **Payload Processing:** The payload JSON field is extracted and set as the Kafka message value²².
- **Header Enrichment:** The metadata fields (e.g., event_type) are mapped to Kafka record headers²².

Because Debezium operates on at-least-once delivery semantics, any connector crash or WAL replay can result in duplicate events being published to Kafka²².

Therefore, downstream consumers must implement the Idempotent Consumer pattern²⁴.

Downstream Deduplication with the Idempotent Consumer Pattern

To protect against duplicate processing from at-least-once deliveries, consumer services must check incoming message IDs against a tracking datastore²⁴.

Under default Hibernate configurations, calling save() registers an entity in the persistence context, but flushes the SQL insert to the database at the end of the transaction lifecycle (transactional write-behind)²⁴.

This deferred write introduces a race condition: if duplicate events are processed concurrently by parallel consumer threads, both threads may see no existing record in the tracking table, proceed to execute duplicate business logic, and only encounter a unique constraint violation when committing the transaction²⁴.

While this rolls back the database changes, any non-transactional side effects (such as external REST API calls) will have already executed twice²⁴.

To resolve this issue, the consumer must execute an immediate database write via saveAndFlush() to acquire a lock on the unique primary key *before* running any downstream business operations²⁴:

```
Java
@Component
public class IdempotentPaymentListener

    private final ProcessedEventRepository processedRepo;
    private final PaymentService paymentService;

    public IdempotentPaymentListener(ProcessedEventRepository processedRepo, PaymentService
paymentService)
    {
        this.processedRepo = processedRepo;
        this.paymentService = paymentService;
    }
}
```

```

    @KafkaListener(topics = "payment.events", groupId = "payment-group")
    @Transactional
    public void onMessage(ConsumerRecord<String, String> record) {
        String eventId = record.key(); // Event ID acts as the unique deduplication key

        try {
            // Force an immediate SQL flush to acquire a database lock and prevent parallel race
            conditions
            processedRepo.saveAndFlush(new ProcessedEvent(eventId));
        } catch (DataIntegrityViolationException e) {
            // Caught unique constraint violation; discard the duplicate and acknowledge the offset
            return;
        }

        // Execute business logic only after successfully securing the database lock
        paymentService.chargeCard(record.value());
    }
}

```

If the business service fails, the transaction rolls back, reverting the ProcessedEvent insertion and allowing Kafka to safely retry the message²⁷.

If the system crashes after the database commits but before Kafka can commit the consumer offset, the message is redelivered on restart²⁷.

During redelivery, the database check throws a DataIntegrityViolationException, prompting the consumer to discard the duplicate and immediately commit the offset, guaranteeing effectively exactly-once execution²⁴.

For low-latency consumer workflows, services can bypass standard primary key checks by storing partition offsets directly alongside business state changes²⁸.

This approach uses a localized offset-tracking table to verify that the incoming offset is greater than the last processed offset for that partition²⁸:

```

Java
@Transactional
public boolean hasAlreadyProcessed(String group, String topic, int partition, long offset) {
    Long lastProcessedOffset = repo.findOffset(group, topic, partition); // Query tracking
    table
    return lastProcessedOffset != null && offset <= lastProcessedOffset; // Returns true if offset
    was processed
}

```

Implementing Exactly-Once Semantics (EOS)

To achieve strict exactly-once semantics for read-process-write loops (where an application consumes from one topic, processes the data, and writes the output to another topic), Kafka combines idempotent producers with transactional writes²⁹.



To enable EOS, `KafkaTemplate` requires enabling producer idempotence and assigning a transaction prefix, which prompts Spring to instantiate a `KafkaTransactionManager` and wire it into the listener containers³¹.

```
YAML
spring:
  kafka:
    producer:
      enable-idempotence: true # Prevents duplicate writes on retry [cite: 30, 31]
      acks: all # Broker waits for all replicas to write before ACK [cite: 24, 31]
      retries: 10
      max-in-flight-requests-per-connection: 1 # Strictly preserves partition sequence [cite: 24, 31]
      transaction-id-prefix: txn-order-proc- # Triggers transaction manager auto-configuration
    consumer:
      enable-auto-commit: false # Disables background thread offset commits
      isolation-level: read_committed # Hides uncommitted transactions from downstream
```

Under this configuration, the consume-transform-produce loop runs atomically³¹. If any stage of the processing pipeline fails, the transaction is aborted, discarding both the outbound messages and the consumer offset commits³⁰:

```
Java
@Service
public class SentenceWordCountService

    private final KafkaTemplate<String, String> kafkaTemplate;

    public SentenceWordCountService(KafkaTemplate<String, String> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    @KafkaListener(topics = "sentences", groupId = "word-count-group")
    public void processSentences(ConsumerRecord<String, String> record, Acknowledgment ack) {
        // Wrap processing inside an atomic Kafka transaction block [cite: 31]
        kafkaTemplate.executeInTransaction(operations -> {
            String[] words = record.value().split(" ");
            for (String word : words) {
                // Publish results using the transactional operations template
                operations.send("word-counts", word, "1");
            }
            // Commit the consumer offsets within the transaction block [cite: 31]
            ack.acknowledge();
        });
        return null;
    }
}
```

Distributed Transaction Design Pattern Comparison

The table below contrasts Kafka Transactions with the Transactional Outbox pattern and application-level offset tracking to guide architectural design decisions.

Architectural	Core	Typical Latency	Primary Disadvantage	Primary
---------------	------	-----------------	----------------------	---------

Pattern	Mechanism	Profile	s	Advantages
Kafka Transactions (EOS)	Uses a centralized Kafka transaction log and coordinator to commit multi-partition writes and offset states atomically ²⁹ .	Moderate (introduces 2-phase commit latency) ²⁴ .	Cannot atomically commit alongside database state changes ²⁴ .	Out-of-the-box support for Kafka-only pipelines with zero database dependencies ³² .
Transactional Outbox with CDC	Writes events directly to a database outbox table in the same local ACID transaction as business changes ²² .	Extremely Low (non-blocking database logs read by Debezium) ²² .	Requires additional infrastructure components (e.g., Debezium) ²⁴ .	Guaranteed consistency across database updates and Kafka events without complex transaction managers ²² .
Deduplication with Offset tracking	Stores Kafka partition offsets directly inside the business database, checking offset state before execution ²⁸ .	Low (requires a quick primary-key database lookup) ²⁸ .	Requires custom state-checking database code on the consumer ²⁸ .	Highly resilient against consumer crashes and partition rebalances ²⁸ .

Fault Tolerance, Retry Topologies, and Error Recovery

Operating high-velocity message pipelines requires robust error recovery patterns to prevent data loss and isolate processing failures²⁷.

Blocking vs. Asynchronous (Non-Blocking) Retries

When a consumer encounters an exception, developers must choose between blocking

partition processing or utilizing asynchronous retry paths³⁶.

```
-- Blocking Retry Pipeline
[Partition Queue] -> [Consumer Attempt 1] -(Fail)-> [Wait] -> [Consumer Attempt 2]
-(Fail)-> [DLQ]
(entire partition is blocked during waiting periods)

-- Asynchronous (Non-Blocking) Retry Pipeline
[Main Topic] -> [Consumer Attempt 1] -(Fail)-> [Publish to Retry Topic] -> [Acknowledge]
Main
[ ]
[ ]
[Retry Topic Consumer] -(Fail)-> [DLQ]
(Main topic continues processing subsequent events without blocking)
```

Blocking Retries

The default error handling strategy in Spring Kafka is to pause consumption from the failing partition and retry processing the message at configured intervals³⁶.

```
Java
@Bean
public DefaultErrorHandler blockingErrorHandler(KafkaTemplate<String, Object> template) {
    // Configures 3 delivery attempts (1 initial + 2 retries) with a 2-second delay
    DefaultErrorHandler handler = new DefaultErrorHandler(
        new DeadLetterPublishingRecoverer(template),
        new FixedBackOff(2000L, 2)
    );
    handler.setLogLevel(KafkaException.Level.WARN); // Prevents log clutter for transient
    failures
    return handler;
}
```

- **Use Case:** Best suited for critical financial ledgers or state-machine updates where strict, logical message sequence ordering must be maintained²⁵.
- **Impact:** If a record fails, the entire partition stops processing³⁶. This halts downstream updates and causes consumer lag to build up²⁷.

Asynchronous (Non-Blocking) Retries

For non-sequenced operations, Spring Kafka supports non-blocking retries³⁶. If a message fails, it is published to a separate, time-delayed retry topic, and the consumer immediately proceeds to the next record in the primary topic³⁶.

```
Java
@RetryableTopic(
    attempts = "3",
    backoff = @Backoff(delay = 1000, multiplier = 2, maxDelay = 10000),
    fixedDelayTopicStrategy = FixedDelayStrategy.SINGLE_TOPIC,
    dltTopicSuffix = ".DLT"
)
@KafkaListener(topics = "notifications", groupId = "notif-group")
public void sendNotification(NotificationEvent event)
{
}
```

- **Use Case:** Ideal for notification dispatch, analytics pipelines, or cache updates where high throughput is prioritized over strict processing order⁸.
- **Impact:** Retried events are processed out-of-order, but the primary topic consumption remains unblocked, avoiding consumer lag spikes³⁶.

Rich Error Context in Dead Letter Queues

When all retry attempts are exhausted, the message is routed to a Dead Letter Queue (DLQ) to preserve data integrity and allow manual investigation¹.

To help operators diagnose the failure, the recovery system should attach rich diagnostic context to the DLQ message headers²⁷.

Spring Kafka's DeadLetterPublishingRecoverer does this by default, adding the failure details directly to the message metadata²⁰:

Java

```
public class DiagnosticDLQRecoverer
```

```
    private final KafkaTemplate<String, Object> kafkaTemplate;
```

```
    public DiagnosticDLQRecoverer(KafkaTemplate<String, Object> kafkaTemplate)
```

```
    {  
        this.kafkaTemplate = kafkaTemplate;  
    }
```

```
    public DeadLetterPublishingRecoverer build()
```

```
    {  
        return new DeadLetterPublishingRecoverer(kafkaTemplate, (record, exception) ->
```

```
        {  
            // Enrich Kafka headers with original message coordinates and exception details
```

```
            record.headers().add("dlq.original.topic", record.topic().getBytes());
```

```
            record.headers().add("dlq.original.partition",
```

```
            ByteBuffer.allocate(4).putInt(record.partition().toArray());
```

```
            record.headers().add("dlq.original.offset",
```

```
            ByteBuffer.allocate(8).putLong(record.offset().toArray());
```

```
            record.headers().add("dlq.error.class", exception.getClass().getName().getBytes());
```

```
            record.headers().add("dlq.error.message", exception.getMessage().getBytes());  
        });
```

```
        // Extract and format the stack trace
```

```
        StringWriter sw = new StringWriter();
```

```
        exception.printStackTrace(new PrintWriter(sw));
```

```
        String stackTrace = sw.toString();
```

```
        record.headers().add("dlq.error.stacktrace", stackTrace.substring(0,
```

```
        Math.min(stackTrace.length(), 1024).getBytes());
```

```
        // Appends the default .DLT suffix to the target topic partition
```

```
        return new TopicPartition(record.topic() + ".DLT", record.partition());  
    }
```

Fatal Error Exemptions

Not all errors should trigger retries³⁶. Transient exceptions, such as database lock timeouts or temporary network outages, are likely to resolve upon retry²⁵.

Conversely, fatal errors, such as serialization failures or missing payload parameters, will fail

consistently³⁵.

Retrying fatal errors wastes system resources and delays alert visibility³⁶.

Both blocking and asynchronous retry handlers should bypass retry loops for these exceptions and route the problematic messages directly to the DLQ²⁷:

```
Java
@Configuration
public class FaultTolerantKafkaConfig

    private final KafkaTemplate<String, Object> kafkaTemplate

    public FaultTolerantKafkaConfig(KafkaTemplate<String, Object> kafkaTemplate)
        this.kafkaTemplate = kafkaTemplate
    }

    @Bean
    public DefaultErrorHandler defaultErrorHandler()
        DefaultErrorHandler handler = new DefaultErrorHandler(
            new DeadLetterPublishingRecoverer(kafkaTemplate),
            new FixedBackOff(1000L, 3)
        )

        // Explicitly register exceptions that bypass retries
        handler.addNotRetriableExceptions(
            DeserializationException.class, // Standard Spring deserialization failures [cite: 20]
            MessageConversionException.class, // JSON mapping mismatches [cite: 20]
            NullPointerException.class, // Logical Null pointers
            IllegalArgumentException.class // Business contract validation failures
        )

        return handler
    }
```

Event-Driven Practice Scenarios for Engineers

These practice scenarios are designed to simulate real-world e-commerce, logistics, and retail pipelines. They focus on application integration patterns, schema design, and failure modes,

rather than infrastructure management.

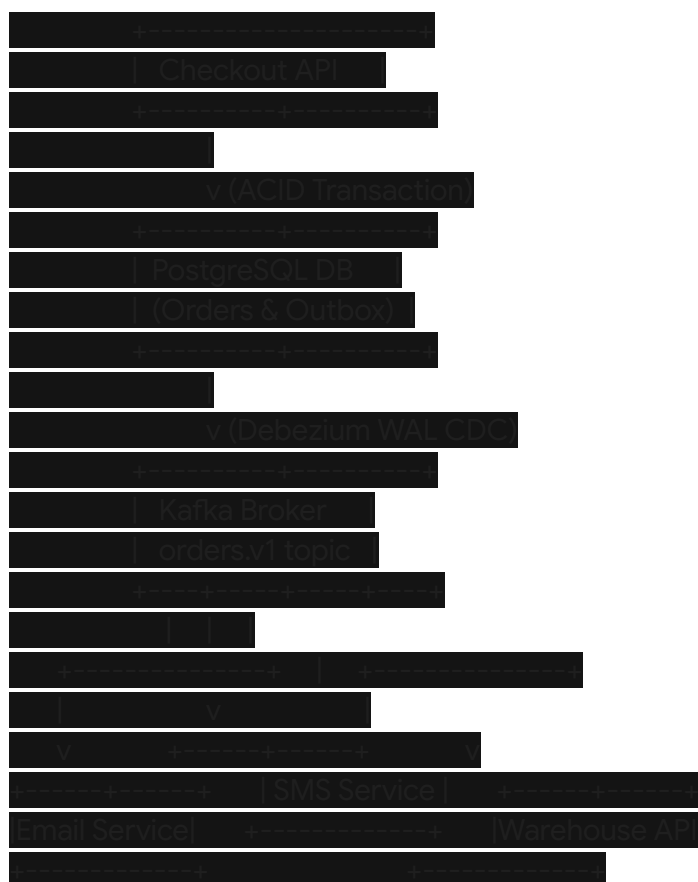
Scenario 1: Decoupled E-Commerce Order Fulfillment & Dynamic Notification Pipeline

Context and Architecture

An e-commerce platform needs to decouple its checkout path from downstream fulfillment and notification services to minimize write latency during peak shopping events¹.

The core checkout API must write the order state to a database and return an immediate HTTP success response to the client browser⁵.

All subsequent processes—including email notifications, SMS alerts, and warehouse updates—are executed asynchronously by independent microservices¹.



To guarantee that events are published only if the order is successfully saved, checkout uses

the Transactional Outbox pattern²².

A Debezium connector captures changes from the outbox table and writes them to the orders.v1 topic in Kafka²².

Downstream consumers read this topic, execute their business logic, and handle potential failures independently¹.

Topic & Payload Specifications

Topic Name	Key Strategy	Value Schema (Protobuf Payload)	Consumer Groups
orders.v1	orderId	message OrderCreated { string orderId = 1; string customerId = 2; double totalAmount = 3; int64 timestamp = 4; } [cite: 1, 15]	email-notification-group, sms-alert-group, warehouse-fulfillment-group [cite: 1]
notifications.email.v1	customerId	message EmailDispatch { string eventId = 1; string customerId = 2; string emailAddress = 3; string templateId = 4; }	email-delivery-group
notifications.sms.v1	customerId	message SmsDispatch { string eventId = 1; string customerId = 2; string phoneNumber = 3; string bodyText = 4; }	sms-delivery-group

Architectural Failure Paths to Solve

- **The Duplicate Ingestion Failure:** Due to network retries, Debezium re-emits duplicate

records for a single order²⁶. The fulfillment consumer must check incoming message IDs against an offset tracking table inside PostgreSQL to ignore duplicates²⁴.

- **External API Outage:** The SMS gateway experiences an outage, throwing a 503 Service Unavailable exception²⁵. The consumer must use an asynchronous retry policy with a progressive backoff to route failed messages to a retry topic, allowing the primary queue to continue processing other SMS alerts³⁶.
- **The Database Transaction Rollback:** The checkout service experiences a validation failure midway through a purchase request³⁴. The transactional context must automatically rollback both the order entity change and the outbox event insert, ensuring no messages are published to Kafka for failed transactions²².

Scenario 2: High-Volume Vehicle Telemetry & Dynamic Surge Pricing Engine

Context and Architecture

A ride-sharing platform must collect real-time GPS telemetry updates from thousands of active driver mobile apps to compute localized demand indexes and update dynamic surge pricing matrices².

Because GPS updates are high-volume and transient, the pipeline must handle large volumes of data without overwhelming relational databases⁷.





Mobile applications push GPS coordinate packets to a specialized Kafka REST Proxy¹⁰.

The proxy validates these events against Protobuf definitions and writes them to a partitioned telemetry.raw topic¹⁰.

A KStream application consumes this topic, aggregates driver density within 10-second windows, and computes a demand multiplier¹⁴.

To optimize storage, the calculated multipliers are compressed with ZSTD and saved directly to Cassandra¹⁵.

Simultaneously, a WebSocket Filtering service monitors these pricing updates and pushes them directly to passenger client browsers in real time¹³.

Topic & Payload Specifications

Topic Name	Key Strategy	Value Schema (Protobuf Payload)	Consumer Groups
telemetry.raw	driverId	message GpsCoordinate { string driverId = 1; double latitude = 2; double longitude = 3; int64 timestamp = 4; } [cite: 2, 15]	telemetry-aggregator-group
pricing.surge-updates	geohash	message SurgeMultiplier { string geohash = 1; double multiplier = 2; int64 calculatedAt = 3; }	surge-caching-group, websocket-streaming-group
websocket.filtered	sessionId	message	websocket-handler

		<pre>FilteredSurge { string sessionId = 1; string geohash = 2; double multiplier = 3; } [cite: 13]</pre>	-group
--	--	--	--------

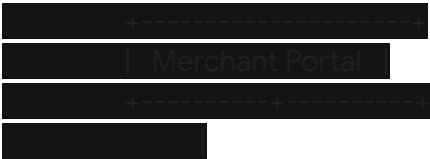
Architectural Failure Paths to Solve

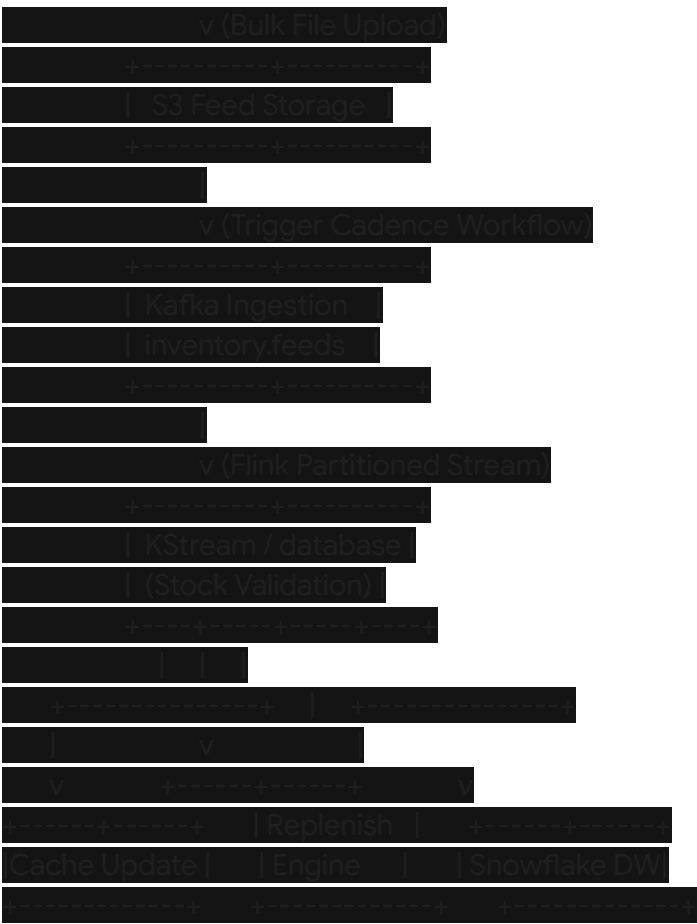
- **The Network Reconnection Spike:** Mobile apps lose signal in tunnels and buffer telemetry updates locally, uploading them as a massive burst once reconnected²⁴. The KStream window aggregator must use a windowing strategy with defined grace periods to drop obsolete coordinates and process delayed events without corrupting demand metrics³⁶.
- **The Telemetry Poison Pill:** A client mobile app publishes a corrupted telemetry packet containing zero coordinates or empty driver IDs²⁷. The consumer must use a custom DeserializationExceptionHandler to isolate and write the bad message to a DLQ, maintaining active consumption across the partition²⁰.
- **Dynamic Geofence Subscriptions:** Users dynamically track surge multipliers as they search different areas on the map¹³. The system must use a PostgreSQL database with notification triggers to alert Kafka Filter Workers to dynamically load and apply these user-defined coordinate filters¹³.

Scenario 3: Asynchronous Merchant Inventory Ingestion & Predictive Stocking Platform

Context and Architecture

A retail platform allows merchant partners to upload bulk inventory update files containing stock level adjustments for thousands of SKUs¹¹. Because updating database columns sequentially for millions of products degrades transaction speeds, the inventory platform handles ingestion asynchronously¹¹.





When a merchant uploads an inventory file, the ingestion service parses the data and publishes SKU mutation events to `inventory.feeds`¹¹.
A Flink job processes this stream, updating the current stock levels in CockroachDB¹¹.
Simultaneously, a machine learning engine reads these stock fluctuations from Kafka to update a predictive model¹¹.
This model identifies potential out-of-stock items based on historic item-not-found patterns and writes replenishment events to Kafka to automate supplier reordering¹¹.

Topic & Payload Specifications

Topic Name	Key Strategy	Value Schema (JSON Payload)	Consumer Groups
inventory.feeds	storeId	{ "feedId": "UUID", "storeId": "STR-40", "items": [{ "sku": "ITEM-1", "quantity": 120, "price": 4.99 }] }	bulk-feed-parser-group

		[cite: 11]	
inventory.mutations	sku	{ "sku": "ITEM-1", "storeId": "STR-40", "newStock": 120, "updateType": "MERCHANT_UPLO AD", "timestamp": 1718290355 }	inventory-caching- group, predictive-stocking -group [cite: 8, 11]
inventory.replenish	supplierId	{ "orderId": "UUID", "sku": "ITEM-1", "quantity": 500, "priority": "CRITICAL" }	supplier-routing-gr oup

Architectural Failure Paths to Solve

- **Database Deadlock Contention:** During high-volume transactional sales, parallel inventory updates attempt to lock the same database rows¹¹. The consumer must implement a non-blocking retry topology, moving failed updates to a retry topic with an exponential backoff to allow database locks to clear²⁵.
- **The Split-Brain Replication Lag:** An inventory update transaction is committed in the primary database, but the read replica remains stale⁹. SpinalTap change-capture listeners must enforce strict sequential offset tracking to ensure updates are processed in transaction commit order⁸.
- **The Infinite Poison-Pill Retry Loop:** A merchant upload contains an unknown currency code that throws an unhandled business validation exception³⁶. The consumer's DefaultErrorHandler must treat this exception as fatal, bypassing retries and writing the record directly to the DLQ to prevent blocking the partition³⁶.

Testing and Automation of Event-Driven Systems

Testing asynchronous systems is challenging because tests cannot easily determine when an event has been processed⁴.

This requires testing frameworks that can mock dependencies, track asynchronous state transitions, and verify delivery semantics⁴.

Functional Verification with Karate

Walmart uses the Karate framework to automate testing for its Spring Kafka microservices³⁹. Because Karate provides built-in assertions and can execute custom Java classes inside test feature files, teams can validate asynchronous flows without needing separate testing

applications³⁹.

```
+-----+
| Karate Feature File |
+-----+
| 1. Reset Offsets via Java helper |
| 2. Publish Event A to Input Topic |
| 3. Poll Output Topic and validate Schema |
+-----+
```

To run clean, isolated tests, the testing suite must reset consumer offsets before each test run, preventing late-arriving messages from previous runs from causing false failures³⁹.

JavaScript

```
// Karate Feature File Segment
```

```
Feature Event-Driven Order Processing Validation
```

```
Scenario Verify Order Placement triggers Payment Dispatch
```

```
# Step 1 Reset the consumer group offset to the latest position |cite: 39
```

```
* java.lang.System.out.println "Resetting automation consumer offsets..."
```

```
* val offsetResetHelper = Java.type 'com.walmart.test.OffsetResetHelper'
```

```
* offsetResetHelper.resetToLatest "payment-group" "order.events" |cite: 39
```

```
# Step 2 Publish a mock order event
```

```
Given url 'http://localhost:8080/orders'
```

```
And request { orderId 'TST-101' customerId 'CST-55' totalAmount 99.50
```

```
When method post
```

```
Then status 202
```

```
# Step 3 Consume and validate the resulting billing packet
```

```
* val kafkaConsumer = Java.type 'com.walmart.test.KafkaTestConsumer'
```

```
* val responsePacket = kafkaConsumer.pollNextMessage "payment.dispatches"
```

```
Then match responsePacket orderId == 'TST-101'
```

```
And match responsePacket.status == 'PENDING'
```

Component Testing with Testcontainers and LocalStack

To verify transactional behaviors, such as the Transactional Outbox pattern or idempotent processing, tests should run against real containerized instances of your infrastructure²⁴. Testcontainers allows developers to programmatically spin up ephemeral Docker containers (e.g., PostgreSQL, Apache Kafka, or LocalStack for AWS mock endpoints) directly within JUnit integration tests²⁴.

```
Java
@SpringBootTest
@Testcontainers
@ActiveProfiles("component-test")
public class OutboxComponentTest

    // Spin up containerized PostgreSQL database
    @Container
    private static final PostgreSQLContainer<?> postgres = new
        PostgreSQLContainer<>("postgres:15-alpine")

    // Spin up containerized Kafka broker
    @Container
    private static final KafkaContainer kafka = new
        KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:7.3.0")

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry)
        // Dynamically bind JDBC and Kafka bootstrap properties to container ports
        registry.add("spring.datasource.url", postgres.getJdbcUrl())
        registry.add("spring.datasource.username", postgres.getUsername())
        registry.add("spring.datasource.password", postgres.getPassword())
        registry.add("spring.kafka.bootstrap-servers", kafka.getBootstrapServers())

    @Autowired
    private OrderFulfillmentService fulfillmentService
```

```

@Autowired
private OutboxRepository outboxRepo;

@Test
public void testConfirmOrderWritesToOutbox() throws Exception {
    UUID orderId = UUID.randomUUID();

    // Execute service call inside local transaction context [cite: 22, 25]
    fulfillmentService.confirmOrder(orderId);

    // Verify that the entity update and outbox record were committed atomically
    List<OutboxEvent> outboxEvents = outboxRepo.findByAggregateId(orderId.toString());
    assertEquals(1, outboxEvents.size());
    assertEquals("OrderConfirmed", outboxEvents.get(0).getEventType());
}

```

This testing pattern ensures that:

1. **Portability:** Component tests can be executed in any environment where Docker is installed, avoiding dependencies on shared dev environments⁴⁰.
2. **State Isolation:** Each test class runs against a clean database instance, reducing test interference⁴⁰.
3. **Behavioral Accuracy:** The system validates real JDBC and Kafka transaction boundaries under realistic conditions before deployment²⁹.

Lightweight Testing with Spring Cloud Stream Test Binder

For functional tests where database integrations are not required, developers can utilize the Spring Cloud Stream Test Binder⁴.

The Test Binder provides an in-JVM mock broker, allowing applications to verify consumer routing and event transformation logic without the network overhead of running a physical Kafka broker⁴.

```

Java
@SpringBootTest
public class PricingStreamTest

@Autowired

```

```

private InputDestination input;

@Autowired
private OutputDestination output;

@Test
public void testSurgePricingPipeline() {
    // Prepare dynamic telemetry coordinate payload
    GpsCoordinate coordinate = GpsCoordinate.newBuilder()
        .setDriverId("DRV-99")
        .setLatitude(37.7749)
        .setLongitude(122.4194)
        .setTimestamp(System.currentTimeMillis())
        .build();

    // Write message directly to the test binder's input channel
    input.send(MessageBuilder.withPayload(coordinate.toByteArray()).build());

    "telemetry.raw"

    // Fetch and assert against resulting price update event
    Message<byte> result = output.receive(5000, "pricing.surge-updates");
    assertNotNull(result);

    SurgeMultiplier multiplier = SurgeMultiplier.parseFrom(result.getPayload());
    assertTrue(multiplier.getMultiplier() == 1.0);
}

```

Using these testing patterns, development teams can build robust automated testing suites that validate everything from basic processing logic to database transactions and complex event delivery flows⁴. This ensures system reliability and maintainability as event-driven architectures scale¹.

Works cited

1. Event-Driven Architecture in 2026: Mastering Kafka with Spring Boot 4 - Medium, <https://medium.com/codetodeploy/event-driven-architecture-in-2026-mastering-kafka-with-spring-boot-4-9751a01f50f5>
2. How Uber uses Apache Kafka in production | Factor House, <https://factorhouse.io/articles/uber-kafka-architecture>
3. Enabling Seamless Kafka Async Queuing with Consumer Proxy - Uber, <https://www.uber.com/us/en/blog/kafka-async-queuing-with-consumer-proxy/>
4. Streaming with Spring Cloud. Hands-on with Spring Cloud Stream | by Bert

Sanders | Walmart Global Tech Blog | Medium,
<https://medium.com/walmartglobaltech/streaming-with-spring-cloud-24a001ad307a>

5. How Kafka Saved Our Payment System And Helped Us Scale to 10 Million Users - Medium,
<https://medium.com/@himanshusingour7/how-kafka-saved-our-payment-system-and-helped-us-scale-to-10-million-users-d0da35d75bc7>
6. Securing Kafka® Infrastructure at Uber,
<https://www.uber.com/us/en/blog/securing-kafka-infrastructure-at-uber/>
7. Building High Throughput Payment Account Processing - Uber,
<https://www.uber.com/us/en/blog/high-throughput-processing/>
8. How Airbnb uses Apache Kafka in production - Factor House,
<https://factorhouse.io/articles/airbnb-kafka-architecture>
9. Building a Next-Generation Key-Value Store at Airbnb,
<https://airbnb.tech/infrastructure/building-a-next-generation-key-value-store-at-airbnb/>
10. Building scalable real time event processing with Kafka and Flink - DoorDash,
<https://careersatdoordash.com/blog/building-scalable-real-time-event-processing-with-kafka-and-flink/>
11. How DoorDash designed a successful write-heavy scalable and reliable inventory platform,
<https://careersatdoordash.com/blog/how-doordash-designed-a-successful-write-heavy-scalable-and-reliable-inventory-platform/>
12. 2022 Summer Intern Projects Article #3 - DoorDash,
<https://careersatdoordash.com/blog/2022-summer-intern-projects-article-3/>
13. Explore near Real-time Streaming Data Using a Web UI | by Ryan Lacerna | Expedia Group Technology | Medium,
<https://medium.com/expedia-group-tech/explore-near-real-time-streaming-data-using-a-web-ui-da116079a74a>
14. Enhancing Data Reliability With An SLO Platform | by Abhishek Chawla | Expedia Group Technology | Medium,
<https://medium.com/expedia-group-tech/enhancing-data-reliability-with-an-slo-platform-de00249756f6>
15. Optimizing API Performance with ZSTD Compression and Protocol Buffers | by Anjana Ganesh | Walmart Global Tech Blog | Medium,
<https://medium.com/walmartglobaltech/optimizing-api-performance-with-zstd-compression-and-protocol-buffers-571ad30f4893>
16. An Approach to Designing a Distributed, Fault-Tolerant, Horizontally Scalable Event Scheduler | by Sandeep Malik | Walmart Global Tech Blog | Medium,
<https://medium.com/walmartglobaltech/an-approach-to-designing-distributed-fault-tolerant-horizontally-scalable-event-scheduler-278c9c380637>
17. Spring Boot Kafka: A Comprehensive Guide - Confluent,
<https://www.confluent.io/learn/spring-boot-kafka/>
18. Building a Spring Boot Kafka Consumer Application: A Step-by-Step Guide - Medium,

- <https://medium.com/@sassiR/building-a-spring-boot-kafka-consumer-application-a-step-by-step-guide-98e8f37ea798>
19. Ignite The Fire In Your Apache Kafka Using Spring Boot | Ksolves,
<https://www.ksolves.com/blog/big-data/nifi/ignite-the-fire-in-your-apache-kafka-using-spring-boot>
 20. Handling Exceptions :: Spring Kafka,
<https://docs.spring.io/spring-kafka/reference/kafka/annotation-error-handling.html>
 21. Spring for Apache Kafka and Protobuf Part 1: Event Data Modeling - Confluent,
<https://www.confluent.io/blog/spring-kafka-protobuf-part-1-event-data-modeling/>
 22. How to Implement the Debezium Outbox Pattern in Spring Boot | Rabinarayan Patra,
<https://www.rabinarayanpatra.com/blogs/implementing-outbox-pattern-cdc-microservices>
 23. Kafka Connect: Transactional Outbox With Debezium: Spring Boot Demo,
<https://www.lydtechconsulting.com/blog/kafka-connect-debezium-demo>
 24. Kafka Idempotent Consumer & Transactional Outbox - Lydtech Consulting,
<https://www.lydtechconsulting.com/blog/kafka-idempotent-consumer-transactional-outbox>
 25. 5 Kafka Error Handling Tips - OSO,
<https://oso.sh/blog/5-kafka-error-handling-tips/>
 26. Debezium + Transaction Outbox in Spring Boot: how to avoid duplicate Kafka events with CDC when reprocessing and multiple consumers? - Stack Overflow,
<https://stackoverflow.com/questions/79886745/debezium-transaction-outbox-in-spring-boot-how-to-avoid-duplicate-kafka-event>
 27. Building Fault-Tolerant Kafka Consumers in Spring Boot - DZone,
<https://dzone.com/articles/building-fault-tolerant-kafka-consumers-in-spring>
 28. Exactly-Once Delivery in Kafka with Spring Boot: | by Dolly - JavaScript in Plain English,
<https://javascript.plainenglish.io/exactly-once-delivery-in-kafka-with-spring-boot-27fdd3cdcd82>
 29. Kafka Transactions: Part 1 - Exactly-Once Messaging,
<https://www.lydtechconsulting.com/blog/kafka-transactions-part-1---exactly-once-messaging>
 30. How to Implement Kafka Exactly-Once Transactions - OneUptime,
<https://oneuptime.com/blog/post/2026-01-30-kafka-exactly-once-transactions/view>
 31. Exactly-Once Delivery in Kafka with Spring Boot — The Final Missing Piece - Stackademic,
<https://blog.stackademic.com/exactly-once-delivery-in-kafka-with-spring-boot-the-final-missing-piece-ecf39d632a35>
 32. Transactions :: Spring Kafka,
<https://docs.spring.io/spring-kafka/reference/kafka/transactions.html>
 33. Exactly Once Processing in Kafka with Java | Baeldung,

- <https://www.baeldung.com/kafka-exactly-once>
34. Transactional Rollback Strategies with Spring Cloud Stream and Apache Kafka, <https://spring.io/blog/2023/10/11/transactional-rollback-strategies-with-spring-cloud-stream-and-apache-kafka/>
 35. Apache Kafka Dead Letter Queue: A Comprehensive Guide - Confluent, <https://www.confluent.io/learn/kafka-dead-letter-queue/>
 36. Handle errors in Spring Kafka consumers like a bliss: retry and DLT reporting for duty | by Vincent AUBRUN | ManoMano Tech team | Medium, <https://medium.com/manomano-tech/handle-errors-in-kafka-consumers-like-a-bliss-retries-and-dlt-reporting-for-duty-dc1ec7cbd50f>
 37. How to Handle Dead Letter Queues in Kafka - OneUptime, <https://oneuptime.com/blog/post/2026-01-24-handle-dead-letter-queues-kafka/view>
 38. Fully Reactive Stream Processing with Apache Kafka and Project Reactor | by Sage Pierce | Expedia Group Technology | Medium, <https://medium.com/expedia-group-tech/fully-reactive-stream-processing-727c4c4371e8>
 39. KAFKA Automation using KARATE. Need for automation: | by Pratyush Shukla | Walmart Global Tech Blog | Medium, <https://medium.com/walmartglobaltech/kafka-automation-using-karate-6a129cfdc210>
 40. Testcontainers & LocalStack for Spring Boot functional tests - Capital One, <https://www.capitalone.com/tech/software-engineering/testcontainers-and-local-stack-functional-testing/>